

construct as program lines. The inclusion of comments and blank lines in the count may encourage developers to introduce artificially many such lines in project development in order to create the illusion of high productivity, which is normally measured in LOC/PM (lines of code/person-month). When comments and blank lines are ignored, the program in Figs. 4.2 contains 17 LOC.

However, there is a fundamental reason for including comments in the program. The quality of comments materially affects maintenance costs because maintenance person will depend on the comments more than anything else to do the job. Conversely, too many blank lines and comments with poor readability and understandability will increase maintenance effort. The problem with including comments is that we must be able to distinguish between useful and useless comments, and there is no rigorous way to do that. Therefore, it is always advisable not to consider comments and blank lines while counting for LOC.

Furthermore, if the main interest is the size of the program for specific functionality, it may be reasonable to include executable statements. The only executable statements in Fig. 4.2 are in lines 5-17 leading to a count of 13. The differences in the counts are 18 to 17 to 13. One can easily see the potential for major discrepancies for large programs with many comments or programs written in languages that allow a large number of descriptive but non-executable statements. Conte [CONT86] has defined lines of code as:

"A line of code is any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program header, declarations, and executable and non-executable statements".

This is the predominant definition for lines of code used by researchers. By this definition, Fig. 4.2 has 18 LOC.

There are some disadvantages of this simple method of counting. LOC is language dependent. A line of assembler is not the same as a line of COBOL. They also reflect what the system is rather than what it does. Measuring systems by the number of lines of code is rather like measuring a building by the number of bricks involved in construction; useful when deciding on the type and number of brick layers but useless in describing the building as a whole. Buildings are normally described in terms of facilities, the number and size of rooms, and their total areas in square feet or meters.

While lines of code have their uses (e.g., in estimating programming time for a program during the build phase of a project), their usefulness is limited for other tasks like functionality, complexity, efficiency, etc. If counting LOC is similar to counting bricks in a building, then what is needed is some way of expressing the system in a way that is analogous to counting the number and size of rooms and their total area in square feet or meters.

4.1.2 Function Count

Measuring software size in terms of lines of code is analogous to measuring a car stereo by the number of resistors, capacitors and integrated circuits involved in its production. The number of components is useful in predicting the number of assembly line staff needed, but it does not say anything about the functions available in the finished stereo. When dealing with customers, the manufacturer talks in terms of functions available (e.g., digital tuning) and not in terms of components (e.g., integrated circuits).

Alan Albrecht while working for IBM, recognised the problem in size measurement in the 1970s, and developed a technique (which he called Function Point Analysis), which appeared to be a solution to the size measurement problem [ALBR79, ALBR83]. It measures functionality from the users point of view, that is, on the basis of what the user requests and receives in return. Therefore, it deals with the functionality being delivered, and not with the lines of code, source modules, files, etc. Measuring size in this way has the advantage that size measure is independent of the technology used to deliver the functions. In other words, two identical counting systems, one written in 4 GL and the other in assembler, would have the same function count. This makes sense to the user, because the object is to buy an accounting system, not lines of assembler and it makes sense to the IT department, because they can measure the performance differences between the assembler and 4GL environments [STEP95].

Function point measures functionality from the users point of view, that is, on the basis of what the user requests and receives in return from the system. The principle of Albrecht's function point analysis (FPA) is that a system is decomposed into functional units

- Inputs : information entering the system.
- Outputs : information leaving the system.
- Enquiries : requests for instant access to information.
- Internal logical files : information held within the system.
- External interface files : Information held by other systems that is used by the system being analyzed.

The FPA functional units are shown in Fig. 4.3.

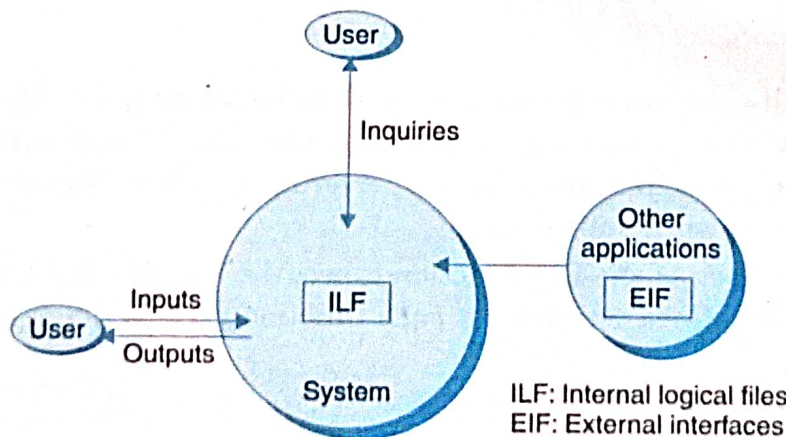


Fig. 4.3: FPA's functional units

The five functional units are divided in two categories:

(i) **Data function types**

- *Internal Logical files (ILF)*: A user identifiable group of logically related data or control information maintained within the system.
- *External Interface files (EIF)*: A user identifiable group of logically related data or control information referenced by the system, but maintained within another system. This means that EIF counted for one system, may be an ILF in another system.

(ii) Transactional function types

- **External Input (EI):** An EI processes data or control information that comes from outside the system. The EI is an elementary process, which is the smallest unit of activity that is meaningful to the end user in the business.
- **External Output (EO):** An EO is an elementary process that generates data or control information to be sent outside the system.
- **External Inquiry (EQ):** An EQ is an elementary process that is made up of an input-output combination that results in data retrieval.

Special features

- Function point approach is independent of the language, tools, or methodologies used for implementation; i.e., they do not take into consideration programming languages, data base management systems, processing hardware or any other data base technology.
- Function points can be estimated from requirement specification or design specifications, thus making it possible to estimate development effort in early phases of development.
- Function points are directly linked to the statement of requirements; any change of requirements can easily be followed by a re-estimate [INCE89].
- Function points are based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring.

This method resolves many of the inconsistencies that arise when using lines of code as a software size measure [MATS94].

Counting function points

The five functional units are ranked according to their complexity i.e., Low, Average, or High, using a set of prescriptive standards. Organisations that use FP methods develop criteria for determining whether a particular entry is Low, Average or High. Nonetheless, the determination of complexity is somewhat subjective.

After classifying each of the five function types, the Unadjusted Function Points (UFP) are calculated using predefined weights for each function type as given in Table 4.1.

Table 4.1: Functional units with weighting factors

Functional units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Table 4.2: UFP calculation table

Functional units	Count complexity			Complexity totals	Functional unit totals
External Inputs (EIs)		Low × 3	=		
		Average × 4	=		
		High × 6	=		
External Outputs (EOs)		Low × 4	=		
		Average × 5	=		
		High × 7	=		
External Inquiries (EQs)		Low × 3	=		
		Average × 4	=		
		High × 6	=		
Internal logical files (ILFs)		Low × 7	=		
		Average × 10	=		
		High × 15	=		
External Interface files (EIFs)		Low × 5	=		
		Average × 7	=		
		High × 10	=		
Total Unadjusted Function Point Count					

The weighting factors are identified (as per Table 4.1) for all functional units and multiplied with the functional units accordingly. The procedure for the calculation of Unadjusted Function Point (UFP) is given in Table 4.2.

The procedure for the calculation of UFP in mathematical form is given below:

$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 Z_{ij} w_{ij}$$

where i indicates the row and j indicates the column of Table 4.1.

w_{ij} : It is the entry of the i^{th} row and j^{th} column of the Table 4.1.

Z_{ij} : It is the count of the number of functional units of Type i that have been classified as having the complexity corresponding to column j .

Organisations that use function point methods develop a criterion for determining whether a particular entry is Low, Average or High. Nonetheless, the determination of complexity is somewhat subjective.

The final number of function points is arrived at by multiplying the UFP by an adjustment factor that is determined by considering 14 aspects of processing complexity which are given in Table 4.3. This adjustment factor allows the UFP count to be modified by at most $\pm 35\%$. The final adjusted FP count is obtained by using the following relationship.

$$FP = UFP * CAF$$

Where CAF is complexity adjustment factor and is equal to $[0.65 + 0.01 \times \sum F_i]$. The F_i ($i = 1$ to 14) are the degrees of influence and are based on responses to questions noted in Table 4.3.

Table 4.3: Computing function points

Rate each factor on a scale of 0 to 5.					
0	1	2	3	4	5
No Influence	Incidental	Moderate	Average	Significant	Essential
Number of factors considered (F_i)					
1. Does the system require reliable backup and recovery?					
2. Is data communication required?					
3. Are there distributed processing functions?					
4. Is performance critical?					
5. Will the system run in an existing heavily utilized operational environment?					
6. Does the system require on line data entry?					
7. Does the on line data entry require the input transaction to be built over multiple screens or operations?					
8. Are the master files updated on line?					
9. Is the inputs, outputs, files, or inquiries complex?					
10. Is the internal processing complex?					
11. Is the code designed to be reusable?					
12. Are conversion and installation included in the design?					
13. Is the system designed for multiple installations in different organisations?					
14. Is the application designed to facilitate change and ease of use by the user?					

Uses of function points

The collection of function point data has two primary motivations. One is the desire by managers to monitor levels of productivity, for example, number of function points achieved per work hour expended. From this perspective, the manager is not concerned with when the function point counts are made, but only that the function points accurately describe the size of the final software project. In this instance, function points have an advantage over LOC in that they provide a more objective measure of software size by which to assess productivity.

Another use of function points is in the estimation of software development cost. There are only a few studies that address this issue, though it is arguably the most important potential use of function point data.

Functions points may compute the following important metrics:

Productivity	=	FP/persons-months
Quality	=	Defects/FP
Cost	=	Rupees /FP
Documentation	=	Pages of documentation per FP

These metrics are controversial and are not universally acceptable. There are standards issued by the International Function Point User Group (IFPUG, covering the Albrecht method) and the United Kingdom Function Point User Group (UFGPU, covering the MK11 method). An ISO standard for function point methods is also being developed.

The function point method continues to be refined. So if we intend to use it we should obtain copies of the latest international function point user group (IFPUG) guidelines and standards. IFPUG is the fastest growing non-profit software metrics user group in the world with an annual growth rate of 30%. Today it has grown to over 800 corporate members and over 1500 individual members in more than 50 countries.

Example 4.1

Consider a project with the following functional units:

Number of user inputs	= 50
Number of user outputs	= 40
Number of user enquiries	= 35
Number of user files	= 06
Number of external interfaces	= 04

Assume all complexity adjustment factors and weighting factors are average.

Compute the function points for the project.

Solution

We know
$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 Z_{ij} w_{ij}$$

$$UFP = 50 \times 4 + 40 \times 5 + 35 \times 4 + 6 \times 10 + 4 \times 7$$

$$= 200 + 200 + 140 + 60 + 28 = 628$$

$$CAF = (0.65 + 0.01 \sum F_i)$$

$$= (0.65 + 0.01(14 \times 3)) = 0.65 + 0.42 = 1.07$$

$$FP = UFP \times CAF$$

$$= 628 \times 1.07 = 672.$$

✓ Example 4.2

An application has the following:

10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries, and a value of complexity adjustment factor of 1.10.

What are the unadjusted and adjusted function point counts ?

Solution

Unadjusted function point counts may be calculated using as:

$$\begin{aligned} \text{UFP} &= \sum_{i=1}^5 \sum_{j=1}^3 Z_{ij} w_{ij} \\ &= 10 \times 3 + 12 \times 7 + 20 \times 7 + 15 + 10 + 12 \times 4 \\ &= 30 + 84 + 140 + 150 + 48 \\ &= 452 \end{aligned}$$

$$\begin{aligned} \text{FP} &= \text{UFP} \times \text{CAF} \\ &= 452 \times 1.10 = 497.2. \end{aligned}$$

Example 4.3

Consider a project with the following parameters.

- (i) External Inputs:
 - (a) 10 with low complexity
 - (b) 15 with average complexity
 - (c) 17 with high complexity.
- (ii) External Outputs:
 - (a) 6 with low complexity
 - (b) 13 with high complexity.
- (iii) External Inquiries:
 - (a) 3 with low complexity
 - (b) 4 with average complexity
 - (c) 2 with high complexity.
- (iv) Internal logical files:
 - (a) 2 with average complexity
 - (b) 1 with high complexity.
- (v) External Interface files:
 - (a) 9 with low complexity.

In addition to above, system requires

- (i) Significant data communication
- (ii) Performance is very critical
- (iii) Designed code may be moderately reusable
- (iv) System is not designed for multiple installations in different organisations.

Other complexity adjustment factors are treated as average. Compute the function points for the project.